
Addressing — Digital interchange models

Adressage — Modèles d'échange numérique

THIS DOCUMENT IS A DRAFT CIRCULATED FOR COMMENT AND APPROVAL. IT IS THEREFORE SUBJECT TO CHANGE AND MAY NOT BE REFERRED TO AS AN INTERNATIONAL STANDARD UNTIL PUBLISHED AS SUCH.

RECIPIENTS OF THIS DRAFT ARE INVITED TO SUBMIT, WITH THEIR COMMENTS, NOTIFICATION OF ANY RELEVANT PATENT RIGHTS OF WHICH THEY ARE AWARE AND TO PROVIDE SUPPORTING DOCUMENTATION.

IN ADDITION TO THEIR EVALUATION AS BEING ACCEPTABLE FOR INDUSTRIAL, TECHNOLOGICAL, COMMERCIAL AND USER PURPOSES, DRAFT INTERNATIONAL STANDARDS MAY ON OCCASION HAVE TO BE CONSIDERED IN THE LIGHT OF THEIR POTENTIAL TO BECOME STANDARDS TO WHICH REFERENCE MAY BE MADE IN NATIONAL REGULATIONS.



Reference Number
ISO/CD 19160-6:2020(E)

© ISO 2020



COPYRIGHT PROTECTED DOCUMENT

© ISO 2020

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
Ch. de Blandonnet 8 • CP 401
CH-1214 Vernier, Geneva, Switzerland
Tel. + 41 22 749 01 11
Fax + 41 22 749 09 47
Email: copyright@iso.org
Website: www.iso.org

Published in Switzerland

Contents

Page

Foreword	v
0 Introduction	vi
0.1 Purpose	vi
0.2 Relationship with other parts of the ISO 19160 standards family	vi
0.3 Address as entered data	vi
0.4 Relationship to the addressable object	vii
1 Scope	1
2 Normative references	1
3 Terms and definitions	1
4 Conformance	3
4.1 General	3
4.2 Address profile register	4
4.3 AddressProfile	4
4.4 AddressClassProfile	4
4.5 AddressComponentProfile	4
4.6 ProfileCompliantAddress	4
4.7 ProfileCompliantAddressComponent	4
4.8 InterchangeAddressClassProfile	4
4.9 FormTemplate	4
4.10 DisplayTemplate	4
5 Address profile registry	4
5.1 General	4
5.2 Roles and responsibilities in the management of an address profile registry	5
5.3 Unmodified ISO 19135-1:2015 requirements	6
5.4 Modified ISO 19135-1:2015 requirements	6
5.5 Specific requirements	7
5.6 Content requirements from ISO 19135-1:2015	7
6 Process of address profile and address interchange	7
6.1 General	7
6.2 Address profiles	7
6.2.1 General	7
6.2.2 Creating address profiles and address class profiles	8
6.2.3 Publishing address profiles	8
6.2.4 Updating address profiles	8
6.2.5 Using address profiles	8
6.2.6 Retiring address profiles	9
6.3 Interchange address	9
6.3.1 General	9
6.3.2 Creating a profile-complaint address	9
6.3.3 Interchange of an interchange address	10
6.3.4 Displaying an interchange address	10
6.3.5 Adding address features	10
6.3.6 Adding associated data to an interchange address	10
6.3.7 Discarding an interchange address	10
7 Address profile model overview	10
8 Data types	13
8.1 Primitive and core data types	13
9 Common models	14
10 Address profile	15
11 Profile compliant address	19

12	InterchangeAddressClassProfile overview.....	22
13	Address feature.....	26
14	Layout template.....	28
15	LayoutLine.....	31
16	Display template.....	31
17	Form template.....	33
Annex A (informative) Abstract test suites.....		36
Annex B (informative) Usage.....		39
Annex C (informative) Examples of objects specified in this document.....		40
Annex D (informative) Examples.....		43
Bibliography.....		51

Foreword

ISO (the International Organization for Standardization) is a worldwide federation of national standards bodies (ISO member bodies). The work of preparing International Standards is normally carried out through ISO technical committees. Each member body interested in a subject for which a technical committee has been established has the right to be represented on that committee. International organizations, governmental and non-governmental, in liaison with ISO, also take part in the work. ISO collaborates closely with the International Electrotechnical Commission (IEC) on all matters of electrotechnical standardization.

The procedures used to develop this document and those intended for its further maintenance are described in the ISO/IEC Directives, Part 1. In particular the different approval criteria needed for the different types of ISO documents should be noted. This document was drafted in accordance with the editorial rules of the ISO/IEC Directives, Part 2 (see www.iso.org/directives).

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. ISO shall not be held responsible for identifying any or all such patent rights. Details of any patent rights identified during the development of the document will be in the Introduction and/or on the ISO list of patent declarations received (see www.iso.org/patents).

Any trade name used in this document is information given for the convenience of users and does not constitute an endorsement.

For an explanation on the voluntary nature of standards, the meaning of ISO specific terms and expressions related to conformity assessment, as well as information about ISO's adherence to the World Trade Organization (WTO) principles in the Technical Barriers to Trade (TBT) see the following URL: www.iso.org/iso/foreword.html.

This document was prepared by Technical Committee ISO/TC 211, *Geographic information / Geomatics*.

A list of all parts in the ISO 19160 series can be found on the ISO website.

This is the first edition of this document.

0 Introduction

0.1 Purpose

Addresses are among the most commonly exchanged information on the Internet, and the interchange of them is crucial to a number of Internet applications, such as electronic commerce, contact exchange, non-postal deliveries, as well as location scheduling.

Yet, addresses can mean much more than just geolocation information:

- as an identity, such as an office address;
- as reference points (waypoints) in routing information; and
- as a delivery point.

The interchange of address information requires a common structure of address data.

For instance, if a software application needs a user to input address information, in order to produce a machine readable address that is interoperable with other systems, the user would need assistance from the application to enter the address in the correct format and structure.

This document specifies a set of data models that focuses on enabling software applications to digitally interchange address profiles, address instances and instructions for their input and display suitable for machine encoding of address information. These models are called the “Address Interchange Object” (“AXO”) models.

Furthermore, a registry mechanism is defined to ensure the address profiles can be made publicly available.

0.2 Relationship with other parts of the ISO 19160 standards family

This standard complements the other parts in the family of ISO 19160 standards:

- ISO 19160-1:2015 describes conceptual models for addressing that allow specification of international address profiles. This document further provides methods to utilize these models in a way suitable for digital interchange and storage.
- ISO 19160-4:2017 defines key terms for postal addressing, postal address components and constraints on their use. It focuses on use cases for postal applications, specifying the methods to write or detect addresses on mail items. This document adapts models from ISO 19160-1:2015 as address interchange models, to facilitate interchange and interaction of addresses between humans and applications, and between applications, and for applications to interact and interchange international addresses, while supporting human input and human-machine interaction, without loss of fidelity.
- ISO 19160-5 focuses on the rendering of addresses for different devices with a focus on their representations on maps, virtual and augmented reality. This document focuses on providing an interoperable data model for the interchange and storage of address profiles and addresses, as well as the input and rendering of compliant addresses in a normalized manner. This approach results into a scheme that can be understood across the spectrum of address consumers, including humans, applications, devices and systems.

0.3 Address as entered data

The lifecycle of an address entry exchanged on the Internet typically starts with manual input of a human actor. This data, structured or unstructured, is then submitted to an Internet-connected application, and

the application may in turn transmit this information to other applications or external parties on behalf of the user who provided the address.

This transmission is usually performed to fulfil service delivery to the user. Within the process, there may be machine-human interactions that require display of the address in human-readable form, as well as machine-to-machine interactions on the address, such as for data validation.

Additional caution shall be placed on the accuracy (or lack thereof) of human input addresses. While an address specified by a human actor may unambiguously distinguish a location, there may be intention or unintentional omissions or additions to an “official” address (if there was one).

0.4 Relationship to the addressable object

Attributes of the object which the address points to are considered out of scope of this document, including the `AddressableObject` model specified in ISO 19160-1:2015.

While considered orthogonal to the AXO models, `AddressableObject` and other similar models can easily be used together with the AXO models.

EXAMPLE In a navigation map, an AXO address can point to an addressable object, which provides a list of its extant facilities.

Addressing — Digital interchange models

1 Scope

This document specifies a set of data models suitable for machine encoding of address information, called the “Address Interchange Object” (“AXO”) models, and the use of them.

Specifically, this document provides:

- data models for digital storage and interchange of address profiles conforming to ISO 19160-1:2015;
- data models for digital storage and interchange of addresses conforming to a specific address profile;
- data models for entry and display templates for entering and displaying addresses conforming to the profile and encoding rules above; and
- the management and operations of a register of address profiles conforming to this document.

Addressability, and features provided within the object which the address points to, is not described in this document. Specifically, the `AddressableObject` from ISO 19160-1:2015 is considered out of scope.

2 Normative references

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO 19160-1:2015, *Addressing — Part 1: Conceptual model*

ISO 19160-3:2020, *Addressing — Part 3: Address data quality*

ISO 19103:2015, *Geographic information — Conceptual schema language*

ISO 19106:2004, *Geographic information — Profiles*

ISO 19115-1:2014, *Geographic information — Metadata — Part 1: Fundamentals*

ISO 19135-1:2015, *Geographic information — Procedures for item registration — Part 1: Fundamentals*

3 Terms and definitions

For the purposes of this document, the terms and definitions given in ISO 19160-1:2015 and the following apply.

ISO and IEC maintain terminological databases for use in standardization at the following addresses:

- ISO Online browsing platform: available at <http://www.iso.org/obp>
- IEC Electropedia: available at <http://www.electropedia.org>

3.1

address display template

address layout template (3.5) for the display of *interchange addresses* (3.12)

3.2

address feature

marking on an interchange address to indicate what it is capable of

3.3

address form template

address layout template of an input form for the entry of interchange addresses

3.4

address interchange object

AXO

digital objects for address interchange defined in ISO 19160-6

3.5

address layout template

specification of layout and positioning of address components for interchange addresses

3.6

address processor

entity that processes interchange addresses

3.7

address profile distributor

entity that distributes interchange address profiles

3.8

data type

specification of a value domain with operations allowed on values in this domain

[SOURCE: ISO 19103:2015, 4.14]

3.9

digital signature

signature

data appended to, or a cryptographic transformation of, a data unit that allows the recipient of the data unit to verify the source and integrity of the data unit

[SOURCE: ISO/IEC 9798-3:2019, 3.3]

3.10

language identifier

language symbol

symbol that uniquely identifies a particular language

[SOURCE: ISO 639-3:2007, 3.3]

3.11

lineage

provenance, source(s) and production process(es) used in producing a resource

[SOURCE: ISO 19115-1:2014, 4.9]

3.12

interchange address

address data compliant to an interchange address class profile

3.13

primitive data type

data type ([3.8](#)) that has no super type

Note 1 to entry: The primitive data type of a data type is the data type itself, if the data type has no super type, and otherwise the primitive data type of the super type of the data type.

[SOURCE: ISO/IEC 10179:1996, 4.23, modified — removal of initial “A”, moved second sentence into Note to entry.]

3.14**private signature key**

private key which defines the private signature transformation

Note 1 to entry: This is sometimes referred to as a secret signature key.

[SOURCE: ISO/IEC 9798-1:2010, 3.23]

3.15**profile**

set of one or more base standards or subsets of base standards, and, where applicable, the identification of chosen clauses, classes, options and parameters of those base standards, that are necessary for accomplishing a particular function

[SOURCE: ISO 19106:2004, 4.5]

3.16**public verification key**

public key which defines the public verification transformation

[SOURCE: ISO/IEC 9798-1:2010, 3.28]

3.17**object identifier**

OID

globally unique value associated with an object to unambiguously identify it

[SOURCE: ISO/IEC TR 29149:2012, 2.11]

3.18**script**

set of graphic characters used for the written form of one or more languages

[SOURCE: ISO 15924:2004, 3.7]

3.19**script code**

combination of characters used to represent the name of a *script* ([3.18](#))

[SOURCE: ISO 15924:2004, 3.8]

3.20**Uniform Resource Identifier**

URI

unique identifier for a resource, structured in conformance with IETF RFC 2396

Note 1 to entry: The general syntax is <scheme>::<scheme-specific-part>. The hierarchical syntax with a namespace is <scheme>://<authority><path>?<query> — see RFC 2396.

[SOURCE: ISO 19136-1:2020, 3.1.62]

4 Conformance**4.1 General**

This part of ISO 19160 defines four classes of requirements and conformance. [Annex A](#) specifies how conformance with these classes shall be tested.

4.2 Address profile register

To conform to this standard, an address profile register shall satisfy all of the requirements specified for a register in ISO 19135-1:2015. See [A.2](#).

4.3 AddressProfile

Any machine-readable description of an ISO 19160-6 `AddressProfile` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.3](#).

4.4 AddressClassProfile

Any machine-readable description of an ISO 19160-6 `AddressClassProfile` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.4](#).

4.5 AddressComponentProfile

Any machine-readable description of an ISO 19160-6 `AddressComponentProfile` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.5](#).

4.6 ProfileCompliantAddress

Any machine-readable description of an ISO 19160-6 `ProfileCompliantAddress` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.6](#).

4.7 ProfileCompliantAddressComponent

Any machine-readable description of an ISO 19160-6 `ProfileCompliantAddressComponent` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.7](#).

4.8 InterchangeAddressClassProfile

Any machine-readable description of an ISO 19160-6 `InterchangeAddressClassProfile` for which conformance is claimed shall pass all the requirements described in the abstract test suite in [A.8](#).

4.9 FormTemplate

Any `FormTemplate` in `InterchangeAddressClassProfile` model for which `FormTemplate` conformance is claimed shall pass the requirements described in the abstract test suite in [A.9](#).

4.10 DisplayTemplate

Any `DisplayTemplate` in `InterchangeAddressClassProfile` model for which `DisplayTemplate` conformance is claimed shall pass the requirements described in the abstract test suite in [A.10](#).

5 Address profile registry

5.1 General

The ISO address profile registry will be managed with version control software, publicly available with an API and a graphical user interface, such as a web-based interface, to satisfy both programatic and human use for the dissemination, display and management of the registry.

The ISO address profile register is a non-hierarchical register. References to principal registers and sub-registers are excluded from this document.

This address profile registry follows the requirements specified in ISO 19135-1:2015, and additional requirements specified in this standard.

Rules for managing a register of geographical information items, including the submission of information, are found in ISO 19135-1:2015, Clause 6.

NOTE Reference to a corresponding ISO 19135-1:2015 requirement is explicitly denoted.

5.2 Roles and responsibilities in the management of an address profile registry

The roles and responsibilities of the register owner, register manager, submitting organizations, control body, registry manager and register user are set out in ISO 19135-1:2015.

Requirement 1 (ISO 19135-1:2015, Requirement 1): The register owner shall set terms and conditions regarding different levels of access to the register and making the contents available to the public. In addition, the register owner shall specify the time period in which the approval process shall be completed.

Requirement 2 (ISO 19135-1:2015, Requirement 2): The register owner shall appoint a register manager. A register owner may serve as the register manager for any register that it has established or it may appoint another organization to serve as the register manager.

Requirement 3 (ISO 19135-1:2015, Requirement 3): The register owner shall decide whether a control body is required for the register and if so appoint the control body. The register owner may serve as the control body for any register that it has established or it may delegate that role to a subgroup within the organization or to the register manager.

Requirement 4 (ISO 19135-1:2015, Requirement 4): A register owner shall specify the criteria that determine which organizations may act as submitting organizations.

Requirement 5 (ISO 19135-1:2015, Requirement 5): The register owner shall clarify the process for a submitting organization to appeal decisions of the control body (if such a body is appointed). The register owner may establish a procedure for such a process. The specification of this procedure shall include appropriate time limits for completion of the process. An alternative solution may be for a submitting organization to resubmit a new proposal with changes or a better justification.

Requirement 6 (ISO 19135-1:2015, Requirement 6): A register manager shall manage a register in conformance with ISO 19135-1:2015, Clause 6.

Requirement 7 (ISO 19135-1:2015, Requirement 7): Upon request, the register manager shall distribute an information package containing a description of the register and how to submit proposals for changes to the content of the register. The information package shall describe what proposed changes to the content may be considered to be substantive.

Requirement 8 (ISO 19135-1:2015, Requirement 8): The register manager shall accept proposals from submitting organizations and manage the proposals as specified in ISO 19135-1:2015, 6.4. The register manager shall pass proposals to the control body for decisions as to acceptability and shall serve as the point of contact between the control body and the submitting organization for negotiations regarding changes to the proposal.

Requirement 9 (ISO 19135-1:2015, Requirement 9): The register manager shall determine whether a submitting organization is qualified in accordance with the criteria established by the register owner.

Requirement 10 (ISO 19135-1:2015, Requirement 10): If a control body is appointed, it shall accept proposals from the register manager and render a decision regarding each proposal within the time limits specified by the register owner.

Requirement 11 (ISO 19135-1:2015, Requirement 11): A registry manager shall ensure the integrity of any register held in the registry and shall provide means for electronic access to the registry for register managers, control body members, and register users.

Requirement 12 (ISO 19135-1:2015, Requirement 12): Register managers shall consider the requirements of different categories of users in selecting methods for publishing the content of a register.

Requirement 13: The ISO address profile register shall have a publicly available record of changes where historical content shall remain publicly available.

Submitting organizations for the ISO address profile register consist of organizations and/or persons responsible for defining and maintaining address profiles.

5.3 Unmodified ISO 19135-1:2015 requirements

The unmodified ISO 19135-1:2015 requirements are as follows:

Requirement 14 (ISO 19135-1:2015, Requirement 13): Every register shall have a technical document describing the item classes to be registered.

NOTE For the ISO address profile register that technical document is this document.

Requirement 15 (ISO 19135-1:2015, Requirement 14): Items shall be individually managed, moving through a set of well-defined states. Information about the temporal history of each item shall be maintained.

Requirement 16 (ISO 19135-1:2015, Requirement 16): A clarification shall not cause any substantive semantic or technical change to a registered item.

Requirement 17 (ISO 19135-1:2015, Requirement 17): Clarification shall be accomplished by updating the existing item in the register. The clarification shall be recorded with a justification of the change and the date on which the register transaction was made.

Requirement 18 (ISO 19135-1:2015, Requirement 19): Retirement shall be accomplished by leaving the item in the register, having its status changed to retired, and including the date on which the register transaction was made.

Requirement 19 (ISO 19135-1:2015, Requirement 21): The register manager shall review proposals received from third parties for completeness and return proposals to the submitting organization if the proposal is incomplete or if the submitting organization is not qualified, else initiate the approval process.

Requirement 20 (ISO 19135-1:2015, Requirement 22): The approval process shall be completed within the time period specified by the register owner.

Requirement 21 (ISO 19135-1:2015, Requirement 23): A registry manager shall ensure that information about valid, invalidated, superseded, or retired items in the register is readily available to users.

5.4 Modified ISO 19135-1:2015 requirements

The ISO 19135-1:2015 requirements modified for this document are as follows:

Requirement 22 (ISO 19135-1:2015, Requirement 15): If an item is superseded by another item, the date the succession occurred shall be captured, along with references to and from the item that superseded it. At any given time, only one item in the series should be “valid”.

NOTE 1 The requirement that only one item in the series is “valid” is removed.

Requirement 23 (ISO 19135-1:2015, Requirement 20): If a register item is deemed to be no longer suitable for the use in the production of new data and has been superseded by a new register item, the original item shall remain in the register, shall have its status changed to superseded, have a reference to the item(s) that superseded it, including the date on which the register transaction was made.

NOTE 2 The option of removing a superseded item from the register is removed.

Requirement 24 (ISO 19135-1:2015, Requirement 18): If an item in a register is found to have substantive error, it shall be left in the register, have its status changed to invalid, have a reference to the item(s) that replaced it, and have the date when the register transaction was made.

NOTE 3 The option of removing an invalidated item from the register is removed.

5.5 Specific requirements

Requirement 25: The ISO address profile registry shall follow ISO 19160-1:2015, Clause 6 in the management of address profiles, including the submission of information.

5.6 Content requirements from ISO 19135-1:2015

The ISO address profile register shall conform to the core register schema in ISO 19135-1:2015, Clause 7. This clause sets out specific requirements as follows:

Requirement 26 (ISO 19135-1:2015, Requirement 24): The core register shall conform to the register schema as specified in UML in ISO 19135-1:2015, Clause 7.

Requirement 27 (ISO 19135-1:2015, Requirement 25): The attribute identifier that designates an item class held in a register that conforms to ISO 19135-1:2015, Clause 7, shall uniquely denote the item class within the context of the register.

Requirement 28 (ISO 19135-1:2015, Requirement 26): The attribute `itemIdentifier` is represented as a `CharacterString` that is used to uniquely denote that item within an item class and is intended for information processing. Once a value has been assigned, it shall not be reused. The class/identifier union shall be unique within the register.

6 Process of address profile and address interchange

6.1 General

Practical usage of AXO models relies on the establishment of an address profile registry.

This clause sets out specific requirements in relation with ISO 19135-1:2015.

The roles and responsibilities of the register owner, register manager, submitting organizations, control body, registry manager and register user are set out in ISO 19135-1:2015, Clause 5.

Data structures that are shared on the address profile registry include address profiles and their metadata in machine-readable format.

A control body is setup to manage creation of and changes to registered items at the registration authority.

6.2 Address profiles

6.2.1 General

```
skinparam Dpi 150
skinparam Monochrome false
hide circle
/' skinparam CircledCharacterRadius 0 '/'
/' skinparam CircledCharacterFontSize 0 '/'
skinparam Default {
    TextAlignment center
    FontName Helvetica
}
skinparam Class {
    AttributeIconSize 0
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
```

```

    FontStyle bold
    StereotypeFontSize 10
}
skinparam Rectangle {
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
    FontStyle bold
    FontSize 11
    StereotypeFontSize 10
}
skinparam Object {
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
    FontStyle bold
    FontSize 11
    StereotypeFontSize 10
}

skinparam Entity {
    StereotypeFontSize 10
}
'***** STATES RELATIONS *****
note top: Creation of an\n interchange\n address profile
(*) --> "Published"
--> "Valid for use"
"Published" --> "Retired"
"Valid for use" --> "Retired"
"Valid for use" -->[Update] "Valid for use"

```

Figure 1 — Lifecycle of an address profile

6.2.2 Creating address profiles and address class profiles

Address class profiles that comply with ISO 19160-1:2015, Annex B, ISO 19106:2004 Conformance class 1, and requirements specified in [Clause 10](#) are created. Address class profiles are created in accordance with requirements specified in [Clause 10](#). Address class profiles are associated with the corresponding address profile in accordance with requirements specified in [Clause 10](#).

6.2.3 Publishing address profiles

Submitting organizations distribute their address profiles to others through direct exchange or through a registry.

6.2.4 Updating address profiles

Submitting organizations can update an address profile and profiles associated with it, and re-distribute it by publishing using a new version number that supersedes the previously published one.

6.2.5 Using address profiles

Applications retrieve suitable address class profiles associated with corresponding address profiles to:

- render address input forms according to the profile-specified form template; or
- display addresses according to the profile-specified display template.

Applications shall consider the validity period of an address class profile, and shall periodically check with the submitting organization (or the registry that the submitting organization distributes via) the latest version of the address profile.

6.2.6 Retiring address profiles

Submitting organizations can indicate the validity period of an address class profile in the profile itself, which if the validity end date has passed, would indicate that the profile is retired.

To immediately retire an address class profile, the submitting organization shall distribute a new version of the address profile with an expired validity period.

6.3 Interchange address

6.3.1 General

```
skinparam Dpi 150
skinparam Monochrome false
hide circle
/' skinparam CircledCharacterRadius 0 '/'
/' skinparam CircledCharacterFontSize 0 '/'
skinparam Default {
    TextAlignment center
    FontName Helvetica
}
skinparam Class {
    AttributeIconSize 0
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
    FontStyle bold
    StereotypeFontSize 10
}
skinparam Rectangle {
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
    FontStyle bold
    FontSize 11
    StereotypeFontSize 10
}
skinparam Object {
    BackgroundColor White
    ArrowColor Black
    BorderColor Black
    FontStyle bold
    FontSize 11
    StereotypeFontSize 10
}

skinparam Entity {
    StereotypeFontSize 10
}

'***** STATES RELATIONS *****'
note top: Creation of an\n interchange\n address instance
(*) --> "Input"
--> "Display"
"Input" --> "Interchange"
"Interchange" --> "Discard"
"Display" --> "Discard"
"Input" -->[Add feature,\n associated data] "Input"
```

Figure 2 — Lifecycle of an interchange address

6.3.2 Creating a profile-complaint address

This clause describes how a profile-compliant address is created. The desired address class profile shall be already retrieved for creating a profile-compliant address that conforms to it.

Typically, a user enters an address through an application interface that implements an input format that conforms to the interchange address class profile's form template, such as an application that runs on an operating system or an Internet application. Such input interface may or may not provide a graphical form.

Conforming applications should not expect general users to be able to input an address with a fully-deduced structure. Immediately after input no features are marked on the interchange address.

6.3.3 Interchange of an interchange address

After a user inputs an address into a structured address form, the user submits this address to the designated recipient, which could be an e-retailer, an electronic business card, or a calendar event.

6.3.4 Displaying an interchange address

The recipient or service that receives an interchange address instance either already has the interchange address class profile or should obtain the interchange address class profile definition. With the interchange address class profile, it could then display the interchange address according to the address class profile's display template.

6.3.5 Adding address features

6.3.5.1 Improving quality of an interchange address

The recipient or service could process the address, such as to further clean up the address by improving the conformance level by fully deducing the address structure, which would add an address feature <https://standards.iso211.org/19160/-6/features/specified> to the address instance.

To ensure that the resulting interchange address with a fully-specified structure is correct, the processor may wish to confirm with the user the resulting address, such as in the case where the address is used for shipping information. This would add the AddressFeature <https://standards.iso211.org/19160/-6/features/confirmed> to the address instance.

6.3.5.2 Verifying an interchange address

The recipient or service could further validate the address, such as with a postal or addressing authority. This would add the AddressFeature <https://verifyingauthority/verified> to the interchange address written by the verifying authority.

6.3.6 Adding associated data to an interchange address

A processor of an interchange address could add extra information such as delivery instructions or routes as associated data.

If the user already has a verified, structured address, then the service could save the effort for verifying the address. Custom address features could be added into the interchange address.

6.3.7 Discarding an interchange address

When a service no longer needs an address, the address should be disposed of.

7 Address profile model overview

The address profile model defined in this document consists of three major layers:

- the AddressProfile model as a container of profiles of addresses and address components,
- the profiles of addresses and address components, and
- the addresses and address components conforming to the profiles of addresses and address components.

The `AddressProfile` model acts as a container of `AddressClassProfiles` and `AddressComponentProfiles`. In general, an `AddressProfile` contains the `AddressClassProfiles` and `AddressComponentProfiles` of a single country or region, which can be represented as country code in ISO 3166-1:2020.

For addresses to conform to this standard, they and their associated address components shall conform to their corresponding `AddressClassProfiles` and `AddressComponentProfiles`.

The association between an address and an address components shall also conform to the `AddressComponentSpecification` defined in between the `AddressClassProfile` and `AddressComponentProfile`. Those conforming addresses and address components are `ProfileCompliantAddresses` and `ProfileCompliantAddressComponents`.

As `ProfileCompliantAddress` and `ProfileCompliantAddressComponent` extend from `Address` and `AddressComponent` respectively, their instances will be compliant to ISO 19160-1:2015 without any compatibility issues.

This document further defines `InterchangeAddressClassProfile` for defining `AddressClassProfiles` which can be claimed as interchangeable. Addresses compliant to `InterchangeAddressClassProfile` will be interchangeable among humans, devices and systems with interoperability.

```

diagram TopDown {
    title 'Address profile model overview'
    caption 'Address profile model overview in UML'
    association {
        owner_type direct
        owner AddressComponent
        member_type aggregation
        member Address
    }
    association {
        owner_type inheritance
        owner AddressComponent
        member_type direct
        member ProfileCompliantAddressComponent
    }
    association {
        owner Address
        member_type direct
        member ProfileCompliantAddressComponent
    }
    association {
        owner_type direct
        owner ProfileCompliantAddressComponent
        member_type direct
        member AddressComponentProfile#profile
    }
    association {
        owner_type aggregation
        owner AddressComponentProfile#addressClassProfile
        member_type direct
        member AttributeProfile#attributeProfile [0..*]
    }
    association {
        owner_type aggregation
        owner AddressProfile#componentProfile [0..*]
        member_type direct
        member AddressComponentProfile
    }
    association {
        owner_type aggregation
        owner AddressClassProfile#addressClassProfile
        member_type direct
    }
}

```

```
    member AttributeProfile#attributeProfile [0..*]
  }
  association {
    owner_type aggregation
    owner AddressProfile#addressProfile [0..*]
    member_type direct
    member AddressClassProfile
  }
  association {
    owner InterchangeAddressClassProfile
    member_type inheritance
    member AddressClassProfile
  }
  association {
    owner_type aggregation
    owner ProfileCompliantAddress
    member_type direct
    member ProfileCompliantAddressComponent
  }
  association {
    owner AddressComponentProfile
    member_type aggregation
    member AddressClassProfile#componentProfile [0..*]
  }
  association {
    owner ProfileCompliantAddress
    member_type inheritance
    member Address
  }
  association {
    owner AddressClassProfile
    member_type direct
    member ProfileCompliantAddress
  }
  association {
    owner AddressComponentSpecification
    member_type direct
    member ProfileCompliantAddressComponent
  }
  association {
    owner ProfileCompliantAddress
    member_type direct
    member AddressClassProfile
  }
  association {
    owner AddressClassProfile
    member_type direct
    member AddressComponentProfile
  }
  association {
    owner AddressComponentSpecification
    member_type direct
    member AddressClassProfile
  }
  association {
    owner AddressComponentProfile
    member_type direct
    member AddressProfile
  }
  class Address {
  }
  association {
    owner_type aggregation
    owner Address
    member_type direct
    member AddressComponent
  }
}
```

}

Figure 3

NOTE This clause defines a number of composite data models, integrates and thereby requires coordination between complex precedent standards, including ISO 19115-1:2014, ISO 19160-1:2015 amongst others.

8 Data types

8.1 Primitive and core data types

These are the common data types used within this document.

Primitive data types (`PrimitiveTypes`) are defined in ISO 19103:2015, 7.2, including:

- `CharacterString`
- `DateTime`, `Date`, `Time`
- `Number`, `Integer`, `Decimal`, `Real`
- `Vector`
- `Boolean`

Core data types (`CoreTypes`) are defined in ISO 19103:2015, A.2.

```

diagram DataTypes {
  title 'Data types'
  caption 'Data types to be used in this standard'
  class iso639Code {
    definition {
      This is the value of language code as specified in <<ISO639-3>>.
    }
  }
  class iso3166Code {
    definition {
      This is the value of country code as specified in <<ISO3166-1>>.
    }
  }
  class iso15924Code {
    definition {
      The type of written script as defined in <<ISO15924>>.
    }
  }
  class iso24229Code {
    definition {
      The type of writing system as defined in <<ISO24229>>.
    }
  }
  class iso19115CI_Date {
    definition {
      The type of `CI_Date` as defined in <<ISO19115-1>>.
    }
  }
  class iso19115PT_Locale {
    definition {
      The type of `PT_Locale` as defined in <<ISO19115-1>>.
    }
  }
  class iso19115EX_Extent {
    definition {
      The geographic, temporal and vertical information of `EX_Extent` defined in <<ISO19115-1, clause=6.6.1>>.
    }
  }
}

```

```

    }
    class iso19115MD_Constraints {
        definition {
            The constraint information of `MD_Constraints` as defined in <<ISO19115-1>>.
        }
    }
    class iso14888Oid {
        definition {
            The public key cryptographic algorithm used for digital signature as defined in
            <<ISO14888-3>>.
        }
    }
}

```

Figure 4

9 Common models

```

diagram CommonModels {
    title 'Common models'
    caption 'Common models to be used in this standard'
    class Localization {
        definition {
            Represents localization metadata of the parent object.

            Realized using a `PT_Locale` object defined in <<ISO19115-1>>,
            a script code from <<ISO15924>>, a writing system code
            from <<ISO24229>>, and indication of text direction.
        }
        +locale: iso19115PT_Locale {
            definition {
                The locale of the parent object.
            }
        }
        +script: iso15924Code {
            definition {
                The type of written script used in the parent object.
            }
        }
        +writingSystem: iso24229Code {
            definition {
                The type of writing system used in the parent object.
            }
        }
        +textDirection: TextDirectionCode {
            definition {
                Indicating in which direction the text of the parent should be read.
            }
        }
    }
    class Validity {
        definition {
            The time interval where an object is considered valid, with a revision number for
            issuance date/time.
        }
        +validityBegins: iso19115CI_Date {
            definition {
                The date and time when validity starts.
            }
        }
        +validityEnds: iso19115CI_Date {
            definition {
                The date and time when validity ends.
            }
        }
    }
}

```

```

}
    +revision: iso19115CI_Date {
    definition {
    Issuance date/time.
    }
}
}
class Signature {
    definition {
    A cryptographic signature used to determine data integrity and validity of the object it
    belongs to.
}
}
    +algorithm: iso14888Oid {
    definition {
    The public key cryptographic algorithm used for this digital signature.
    }
}
}
    +publicKey: Uri {
    definition {
    A reference to the actual public key used to verify the digital signature, a URI where
    the public key of the signer used for this signature is found.
    }
}
}
    +signature: CharacterString {
    definition {
    The actual digital signature value encoded in Base64 format.
    }
}
}
enum TextDirectionCode {
    definition {
    An enumeration value of TextDirectionCode represents the reading
    direction of textual data whether it is from left to right or right to
    left, and from top to bottom or bottom to top.
}
}
    leftToRightTopToBottom{
    definition {
    Indicating that text should be read left to right, and top to bottom.
    }
}
}
    rightToLeftTopToBottom{
    definition {
    Indicating that text should be read right to left, and top to bottom.
    }
}
}
    leftToRightBottomToTop{
    definition {
    Indicating that text should be read left to right, and bottom to top.
    }
}
}
    rightToLeftBottomToTop{
    definition {
    Indicating that text should be read right to left, and bottom to top.
    }
}
}
}
}

```

Figure 5

10 Address profile

```

diagram AddressProfile {
    title 'Address profile'

```

```
caption 'Address profile data model'
association {
    owner AddressProfile
    member_type direct
    member AddressClassProfile
}
association {
    owner AddressComponentProfile
    member_type direct
    member AddressClassProfile
}
class AddressProfile {
    +country: iso3166Code [0..*]{
definition {
The country of which this AddressProfile represents.
}
}
}
association {
    owner_type aggregation
    owner AddressProfile#componentProfile [0..*]
    member_type direct
    member AddressComponentProfile
}
class AddressClassProfile {
    definition {
        An `AddressClassProfile` represents a profile of a `ProfileCompliantAddress`.
        It corresponds to the concept of an `AddressClass` originally
        expressed in <<ISO19160-1>> as a referenced codelist.
```

It represents additional attributes and restrictions to the
`ProfileCompliantAddress` model. It also represents metadata for the use of
the profile.

A single profile can include and support multiple types of
address formats, such as a numbered street address and a
"PO Box" address simultaneously. Each of these address formats
is represented as an interchange address class.

[example]

--
The US Numbered Thoroughfare Address with this syntax can be
represented as an address class profile:

```
*\{sp}\{\ Complete Landmark Name or Complete Place Name \} +
*\{sp}\{\ Complete Address Number * \} +
*\{sp}\{\ Complete Street Name * \} +
*\{sp}\{\ Complete Subaddress \} +
*\{sp}\{\ Complete Place Name * \} +
*\{sp}\{\ State Name * \} +
*\{sp}\{\ Zip Code \} +
*\{sp}\{\ Zip Plus 4 \} +
*\{sp}\{\ Country Name \}
```

```
--
}
    +id: CharacterString {
        definition {
            Unique identifier of this AddressClassProfile.
        }
    }
    +type: CharacterString {
        definition {
            Intended usage of this profile.
        }
    }
    +description: CharacterString {
```

```

definition {
  Textual description of this profile.
}
}
+localization: Localization {
  definition {
    The language and script used within this profile.
  }
}
+signature: Signature [0..1]{
  definition {
    The digital signature to verify the integrity of this profile, and the identity of the
    publishing authority.
  }
}
+areaApplicability: iso19115MD_SpatialRepresentation [0..*]{
  definition {
    The geographic representation of which this AddressClassProfile applies to. Overlapping
    geographic areas are allowed across different profiles.
  }
}
+timeToLive: Integer {
  definition {
    The maximum time interval between refreshing of this profile via an authoritative
    source, in seconds.
  }
}
+validity: Validity {
  definition {
    The period of date and time that this profile should be considered as valid.
  }
}
}
association {
  owner_type aggregation
  owner AddressClassProfile#addressClassProfile
  member_type direct
  member AttributeProfile#attributeProfile [0..*]
}
association {
  owner AddressClassProfile
  member_type direct
  member ProfileCompliantAddress
}
class AddressComponentProfile {
  definition {
    An `AddressComponentProfile` represents a profile of a
    `ProfileCompliantAddressComponent`,
    which is extended from the `AddressComponent` model defined in <<ISO19160-1>>.

```

It represents additional attributes and restrictions to the `Address` model of a `ProfileCompliantAddress`. It also represents metadata for the use of the profile.

```

}
+key: CharacterString {
  definition {
    An identifier of this `AddressComponentProfile`, shall be unique within the
    `AddressProfile`.
  }
}
+description: CharacterString {
  definition {
    Textual description of this component.
  }
}
+example: CharacterString [0..1]{

```

```

definition {
  A textual example to demonstrate the correct use of this component.
}
}
}
association {
  owner_type aggregation
  owner AddressComponentProfile#addressClassProfile
  member_type direct
  member AttributeProfile#attributeProfile [0..*]
}
association {
  owner AddressComponentProfile
  member_type direct
  member ProfileCompliantAddressComponent#profile [1..1]
}
class AddressComponentSpecification {
  definition {
    An `AddressComponentSpecification` specifies the cardinalities between a
    `ProfileCompliantAddress` and `ProfileCompliantAddressComponents`.

A `ProfileCompliantAddress` MUST conform to this
`AddressComponentSpecification` to specifies its components.

}
  +maxCardinality: Integer {
    definition {
      The maximum number of instances of the specific `ProfileCompliantAddressComponent` of a
      `ProfileCompliantAddress`.
    }
  }
  +minCardinality: Integer {
    definition {
      The minimum number of instances of the specific `ProfileCompliantAddressComponent` of a
      `ProfileCompliantAddress`.
    }
  }
}
class AttributeProfile {
  definition {
    An `AttributeProfile` represents name, cardinalities and value type of an
    attribute in a model that complies with a profile that includes this
    AttributeProfile.

If it represents an attribute that has been previously defined, e.g. an
attribute in Address model defined in <<ISO19160-1>>, it extends the
definition of the attribute by redefining the name, cardinalities and value
type.

}
  +name: CharacterString {
    definition {
      The name of the attribute represented by this AttributeProfile.
    }
  }
  +minCardinality: Integer [0..1]{
    definition {
      The minimum number of occurrences of the attribute represented by this AttributeProfile.
    }
  }
  +maxCardinality: Integer [0..1]{
    definition {
      The maximum number of occurrences of the attribute represented by this AttributeProfile.
    }
  }
  +valueType: CharacterString [0..1]{
    definition {

```

```

    The data type of values of the attribute represented by this AttributeProfile.
  }
}
}
}

```

Figure 6

11 Profile compliant address

```

diagram ProfileCompliantAddress {
  title 'Profile compliant address'
  caption 'Profile compliant address data model'
  association {
    owner AddressClassProfile
    member_type direct
    member AddressComponentProfile
  }
  association {
    owner ProfileCompliantAddress
    member_type direct
    member ProfileCompliantAddressComponent
  }
  association {
    owner Address
    member_type direct
    member AddressComponent
  }
  class AddressClassProfile {
    definition {
      An `AddressClassProfile` represents a profile of a `ProfileCompliantAddress`.
      It corresponds to the concept of an `AddressClass` originally
      expressed in <<ISO19160-1>> as a referenced codelist.
    }
  }
}

```

It represents additional attributes and restrictions to the `ProfileCompliantAddress` model. It also represents metadata for the use of the profile.

A single profile can include and support multiple types of address formats, such as a numbered street address and a "PO Box" address simultaneously. Each of these address formats is represented as an interchange address class.

It represents additional attributes and restrictions to the `ProfileCompliantAddress` model. It also represents metadata for the use of the profile.

A single profile can include and support multiple types of address formats, such as a numbered street address and a "PO Box" address simultaneously. Each of these address formats is represented as an interchange address class.

[example]

--

The US Numbered Thoroughfare Address with this syntax can be represented as an address class profile:

```

*{\sp\\}\{ Complete Landmark Name or Complete Place Name \} +
*{\sp\\}\{ Complete Address Number * \} +
*{\sp\\}\{ Complete Street Name * \} +
*{\sp\\}\{ Complete Subaddress \} +
*{\sp\\}\{ Complete Place Name * \} +
*{\sp\\}\{ State Name * \} +
*{\sp\\}\{ Zip Code \} +
*{\sp\\}\{ Zip Plus 4 \} +
*{\sp\\}\{ Country Name \}
--

}
  +id: CharacterString {
    definition {
      Unique identifier of this AddressClassProfile.
    }
  }
  +type: CharacterString {

```

```

definition {
  Intended usage of this profile.
}
}
+description: CharacterString {
  definition {
    Textual description of this profile.
  }
}
+localization: Localization {
  definition {
    The language and script used within this profile.
  }
}
+signature: Signature [0..1]{
  definition {
    The digital signature to verify the integrity of this profile, and the identity of the
    publishing authority.
  }
}
+areaApplicability: iso19115MD_SpatialRepresentation [0..*]{
  definition {
    The geographic representation of which this AddressClassProfile applies to. Overlapping
    geographic areas are allowed across different profiles.
  }
}
+timeToLive: Integer {
  definition {
    The maximum time interval between refreshing of this profile via an authoritative
    source, in seconds.
  }
}
+validity: Validity {
  definition {
    The period of date and time that this profile should be considered as valid.
  }
}
}
association {
  owner_type aggregation
  owner AddressClassProfile#addressClassProfile
  member_type direct
  member AttributeProfile#attributeProfile [0..*]
}
association {
  owner AddressClassProfile
  member_type direct
  member ProfileCompliantAddress
}
class AddressComponentProfile {
  definition {
    An `AddressComponentProfile` represents a profile of a
    `ProfileCompliantAddressComponent`,
    which is extended from the `AddressComponent` model defined in <<ISO19160-1>>.

```

It represents additional attributes and restrictions to the `Address` model of a `ProfileCompliantAddress`. It also represents metadata for the use of the profile.

```

}
+key: CharacterString {
  definition {
    An identifier of this `AddressComponentProfile`, shall be unique within the
    `AddressProfile`.
  }
}
+description: CharacterString {

```

```

definition {
  Textual description of this component.
}
}
+example: CharacterString [0..1]{
definition {
  A textual example to demonstrate the correct use of this component.
}
}
}
association {
  owner AddressComponentProfile
  member_type aggregation
  member AddressClassProfile#componentProfile [0..*]
}
association {
  owner_type aggregation
  owner AddressComponentProfile#addressClassProfile
  member_type direct
  member AttributeProfile#attributeProfile [0..*]
}
association {
  owner AddressComponentProfile
  member_type direct
  member ProfileCompliantAddressComponent#profile [1..1]
}
class ProfileCompliantAddress {
  definition {
    ProfileCompliantAddress extends from the `Address` model defined in
    <<ISO19160-1>> to represent an `Address` complying with an
    `AddressClassProfile`.

```

A `ProfileCompliantAddress` MUST conform to the constraints and requirements specified in `AddressClassProfile`, and its associated profiles and specifications.

```

}
}
association {
  owner ProfileCompliantAddress
  member_type inheritance
  member Address
}
class ProfileCompliantAddressComponent {
  definition {
    ProfileCompliantAddressComponent extends from the AddressComponent model
    defined in <<ISO19160-1>> to represent an Address complying with an
    AddressClassProfile.

```

A ProfileCompliantAddressComponent MUST conform to the constraints and requirements specified in AddressComponentProfile, and its associated profiles and specifications.

```

}
}
association {
  owner ProfileCompliantAddressComponent
  member_type inheritance
  member AddressComponent
}
class AddressComponentSpecification {
  definition {
    An `AddressComponentSpecification` specifies the cardinalities between a
    `ProfileCompliantAddress` and `ProfileCompliantAddressComponents`.

```

A `ProfileComplinatAddress` MUST conform to this `AddressComponentSpecification` to specifies its components.

```

}
    +maxCardinality: Integer {
    definition {
    The maximum number of instances of the specific `ProfileCompliantAddressComponent` of a
    `ProfileCompliantAddress`.
    }
}
    +minCardinality: Integer {
    definition {
    The minimum number of instances of the specific `ProfileCompliantAddressComponent` of a
    `ProfileCompliantAddress`.
    }
}
}
}
class Address {
}
class AddressComponent {
}
}
}

```

Figure 7

12 InterchangeAddressClassProfile overview

```

diagram InterchangeAddressClassProfile {
    title 'InterchangeAddressClassProfile overview'
    caption 'InterchangeAddressClassProfile model'
    association {
        owner InterchangeAddressClassProfile
        member_type direct
        member AddressClassProfile
    }
    association {
        owner FormTemplate
        member_type direct
        member DisplayTemplate
    }
    association {
        owner DisplayTemplate
        member_type direct
        member AttributeProfileSignature
    }
    association {
        owner AttributeProfileSignature
        member_type direct
        member AttributeProfileAddressFeature
    }
    association {
        owner AttributeProfileAddressFeature
        member_type direct
        member AttributeProfileValidity
    }
    association {
        owner InterchangeAddressClassProfile
        member_type direct
        member AttributeProfileSignature
    }
    association {
        owner AttributeProfileAddressFeature
        member_type direct
        member AttributeProfile
    }
}
class AddressClassProfile {
    definition {
    An `AddressClassProfile` represents a profile of a `ProfileCompliantAddress`.

```

It corresponds to the concept of an `AddressClass` originally expressed in <<ISO19160-1>> as a referenced codelist.

It represents additional attributes and restrictions to the `ProfileCompliantAddress` model. It also represents metadata for the use of the profile.

A single profile can include and support multiple types of address formats, such as a numbered street address and a "PO Box" address simultaneously. Each of these address formats is represented as an interchange address class.

[example]

--

The US Numbered Thoroughfare Address with this syntax can be represented as an address class profile:

```
*\{sp}\{\ Complete Landmark Name or Complete Place Name \} +
*\{sp}\{\ Complete Address Number * \} +
*\{sp}\{\ Complete Street Name * \} +
*\{sp}\{\ Complete Subaddress \} +
*\{sp}\{\ Complete Place Name * \} +
*\{sp}\{\ State Name * \} +
*\{sp}\{\ Zip Code \} +
*\{sp}\{\ Zip Plus 4 \} +
*\{sp}\{\ Country Name \}
--

}
  +id: CharacterString {
    definition {
      Unique identifier of this AddressClassProfile.
    }
  }
  +type: CharacterString {
    definition {
      Intended usage of this profile.
    }
  }
  +description: CharacterString {
    definition {
      Textual description of this profile.
    }
  }
  +localization: Localization {
    definition {
      The language and script used within this profile.
    }
  }
  +signature: Signature [0..1]{
    definition {
      The digital signature to verify the integrity of this profile, and the identity of the publishing authority.
    }
  }
  +areaApplicability: iso19115MD_SpatialRepresentation [0..*]{
    definition {
      The geographic representation of which this AddressClassProfile applies to. Overlapping geographic areas are allowed across different profiles.
    }
  }
  +timeToLive: Integer {
    definition {
      The maximum time interval between refreshing of this profile via an authoritative source, in seconds.
    }
  }
}
```

```

    +validity: Validity {
definition {
The period of date and time that this profile should be considered as valid.
}
}
}
association {
    owner_type aggregation
    owner AddressClassProfile#addressClassProfile
    member_type direct
    member AttributeProfile#attributeProfile [0..*]
}
association {
    owner AddressClassProfile
    member_type direct
    member ProfileCompliantAddress
}
class InterchangeAddressClassProfile {
    definition {
        This `InterchangeAddressClassProfile` extends from `AddressClassProfile` to
        enforce the existence of several subclasses of `AttributeProfile` for
        interchange.

```

It also introduces `FormTemplate` and `DisplayTemplate` to allow the display, entry of, and the interchange of an `ProfileCompliantAddress` (address instance complying with an `InterchangeAddressClassProfile`).

```

}
}
association {
    owner_type aggregation
    owner InterchangeAddressClassProfile#attributeProfile
    member_type direct
    member AttributeProfileAddressFeature#addressClassProfile
}
association {
    owner_type aggregation
    owner InterchangeAddressClassProfile#attributeProfile
    member_type direct
    member AttributeProfileValidity#addressClassProfile
}
association {
    owner_type aggregation
    owner InterchangeAddressClassProfile
    member_type direct
    member FormTemplate#formTemplate [0..*]
}
association {
    owner_type aggregation
    owner InterchangeAddressClassProfile
    member_type direct
    member DisplayTemplate#displayTemplate [0..*]
}
class DisplayTemplate {
    definition {
        The display template is used for displaying an
        interchange address instance belonging to an interchange
        address class. Generally, one address class is represented
        by one display template.
    }
}
}
association {
    owner DisplayTemplate
    member_type inheritance
    member LayoutTemplate

```

```

}
association {
    owner_type composition
    owner DisplayTemplate
    member_type direct
    member DisplayLine#formLine [1..*]
}
class FormTemplate {
    definition {
        This section shows how an input form can be rendered according to
        the interchange address class.
    }
}

```

The form template should use should also provide an example for display purposes in input (e.g., Help section).

The form template is used for displaying an input form for entry of address data into an interchange address instance. Generally, one address class is represented by one input template.

NOTE: This part has no equivalence in PATDL.

```

}
}
association {
    owner FormTemplate
    member_type inheritance
    member LayoutTemplate
}
association {
    owner_type composition
    owner FormTemplate
    member_type direct
    member FormLine#formLine [1..*]
}
class Signature {
    definition {
        AttributeProfileSignature extends from AttributeProfile to represent the signature
        attribute of the object it belongs to.
    }
}
}
association {
    owner Signature
    member_type inheritance
    member AttributeProfile
}
class AddressFeature {
    definition {
        AttributeProfileAddressFeature extends from AttributeProfile to represent the
        feature attribute of the object it belongs to.
    }
}
}
association {
    owner AddressFeature
    member_type inheritance
    member AttributeProfile
}
class Validity {
    definition {
        AttributeProfileValidity extends from AttributeProfile to represent the validity
        attribute of the object it belongs to.
    }
}
}
association {
    owner Validity
}

```

```

    member_type inheritance
    member AttributeProfile
}
class AttributeProfile {
    definition {
        An `AttributeProfile` represents name, cardinalities and value type of an
        attribute in a model that complies with a profile that includes this
        AttributeProfile.
    }
}

```

If it represents an attribute that has been previously defined, e.g. an attribute in Address model defined in <<ISO19160-1>>, it extends the definition of the attribute by redefining the name, cardinalities and value type.

```

}
    +name: CharacterString {
    definition {
        The name of the attribute represented by this AttributeProfile.
    }
}
    +minCardinality: Integer [0..1]{
    definition {
        The minimum number of occurrences of the attribute represented by this AttributeProfile.
    }
}
    +maxCardinality: Integer [0..1]{
    definition {
        The maximum number of occurrences of the attribute represented by this AttributeProfile.
    }
}
    +valueType: CharacterString [0..1]{
    definition {
        The data type of values of the attribute represented by this AttributeProfile.
    }
}
}
}
}
}

```

Figure 8

13 Address feature

```

diagram AddressFeature {
    title 'Address feature'
    caption 'Address feature data model'
    class AddressFeature {
        definition {
            Address feature represents the status of a particular aspect of an
            interchange address instance, as determined by an address processor.
            Each of the aspects may be differentiated by multiple statuses.
        }
    }
}

```

This is an abstract class.

To represent specific features, this class should be extended upon.

```

}
    +signature: Signature [0..1]{
    definition {
        Cryptographic signature used to ensure that the
        feature is marked by an address process trusted for
        verification of this feature.
        The signature generated shall incorporate the `id` attribute
        of the interchange address instance that owns it.
    }
}
}
    class SpecificationLevel {

```

```
definition {
```

When an address is first manually entered by a common person, it is unrealistic to expect this person to fully adhere to the defined structure.

The allowed values of specification levels are:

* Fully structured, all components are accurately separated and cannot be further split.

* Partially structured, some components may be structurally correct, but some other components are still unstructured.

* Unstructured, the user has entered free-form text with no regard of structuring them.

[example]

Digital addresses entered on e-commerce sites and address books are often partially structured, with defined country, region and city, but with street addresses often unstructured.

[example]

Digital addresses for utility installations (e.g. electricity, water supplies) are often fully structured.

Therefore, an organization who accepts these addresses may wish to re-structure them or fill in any missing address components.

```
}
    +value: SpecificationLevelValue {
    definition {
    Indicating the extent of specification of an address instance on its structure.
    }
}
}
    association {
        owner SpecificationLevel
        member_type inheritance
        member AddressFeature
    }
    class QualityStatus {
        definition {
        An address entered may not have been validated for correctness.
        An address processor, such as a postal handling entity,
        may decide to validate the address to a certain degree
        in order to determine the correctness of the address.
        }
    }
    +value: QualityCode {
    definition {
    Indicating the quality of an address instance.
    }
}
}
    association {
        owner QualityStatus
        member_type inheritance
        member AddressFeature
    }
    class Geolocation {
        definition {
        An address is commonly linked to one or more geolocations.
        The value of a location is given by `EX_Extent` defined
        in <<ISO19115-1,clause=6.6.1>>, which supports
        geographic, temporal and vertical specification.
        }
}
```

```

    +value: iso19115EX_Extent {
    definition {
    The geographic, temporal and vertical information of the location defined in <<ISO19115-
1,clause=6.6.1>>.
    }
    }
    association {
    owner Geolocation
    member_type inheritance
    member AddressFeature
    }
    enum SpecificationLevelValue {
    unstructured{
    definition {
    The user has entered free-form text with no regard of structuring them.
    }
    }
    partial{
    definition {
    Some components may be structurally correct, but some other components are still
unstructured.
    }
    }
    full{
    definition {
    All components are accurately separated and cannot be further split.
    }
    }
    enum QualityCode {
    confirmedCorrect{
    definition {
    Indicating that the address is confirmed to be correct to the best of knowledge of the
address processor.
    }
    }
    mostlyCorrect{
    definition {
    Indicating with high confidence that the address is correct.
    }
    }
    possiblyCorrect{
    definition {
    Indicating with confidence that the address is correct.
    }
    }
    structurallyCorrect{
    definition {
    Indicating that the address components have the correct value types.
    }
    }
    unknown{
    definition {
    Indicating that this address has not been validated in any way, and it cannot be
assigned a quality.
    }
    }
    }
    }

```

Figure 9**14 Layout template**

```

diagram LayoutTemplate {

```

```

title 'Layout template'
caption 'Layout template data model'
class LayoutTemplate {
  definition {
    The interchange layout template serves as the basis of the
    interchange display template and
    the interchange form template.
  }
}

```

A layout template is designed to present an interchange address instance in the case of a display template, or present an input form for entry of an interchange address instance in the case of an interchange form template.

The interchange layout template is based on the following assumptions:

- * the resulting layout is displayed in a bounded rectangular area
- * the entire template is in the same script and locale.

The action of inserting components from a given interchange address instance into a layout template is called "render".

// TODO: define "render" in terms and definition

[example]

A postal mail label can be conceptually considered as an interchange address instance rendered according to a layout template.

```

}
+id: CharacterString {
  definition {
    Unique identifier for the layout template.
  }
}
+name: CharacterString {
  definition {
    Descriptive name of the layout template.
  }
}
+description: CharacterString {
  definition {
    Textual description of the layout template.
  }
}
+localization: Localization {
  definition {
    Locale and script information of the layout template.
  }
}
+dimension: LayoutDimension [0..*]{
  definition {
    Physical dimensions of the rectangular bounding box for the rendered layout output.
  }
}
+orientation: LayoutOrientation [0..1]{
  definition {
    Horizontal or vertical of the text orientation.
  }
}
+lines: LayoutLine [0..*]{
  definition {
    The constituent parts of this LayoutTemplate.
  }
}
}
class LayoutDimension {
  definition {

```

Physical dimensions of the rectangular bounding box for the rendered layout output.

```

}
+width: Integer [0..1]{
definition {
The horizontal length of the rectangular bounding box for the rendered layout output.
}
}
+height: Integer [0..1]{
definition {
The vertical length of the rectangular bounding box for the rendered layout output.
}
}
enum LayoutOrientation {
definition {
The enumeration value to indicate the orientation of the address rendered by the
LayoutTemplate.
}
horizontal{
definition {
Indicating that the address will be rendered from left to right or right to left.
}
}
vertical{
definition {
Indicating that the address will be rendered from top to bottom or bottom to top.
}
}
}
class DisplayTemplate {
definition {
The display template is used for displaying an
interchange address instance belonging to an interchange
address class. Generally, one address class is represented
by one display template.
}
}
association {
owner DisplayTemplate
member_type inheritance
member LayoutTemplate
}
association {
owner_type composition
owner DisplayTemplate
member_type direct
member DisplayLine#formLine [1..*]
}
class FormTemplate {
definition {
This section shows how an input form can be rendered according to
the interchange address class.

```

The form template should use should also provide an example for display purposes in input (e.g., Help section).

The form template is used for displaying an input form for entry of address data into an interchange address instance. Generally, one address class is represented by one input template.

NOTE: This part has no equivalence in PATDL.

```

}
}
association {

```

```

    owner FormTemplate
    member_type inheritance
    member LayoutTemplate
}
association {
    owner_type composition
    owner FormTemplate
    member_type direct
    member FormLine#formLine [1..*]
}
}

```

Figure 10

15 LayoutLine

```

diagram LayoutLine {
    title 'Layout line'
    caption 'Layout line data model'
    class LayoutLine {
        definition {
            A layout line represents a line in a rendered address display
            or an address input form.

```

It forms the basis of the address display line and address form line.

```

    }
    +dimensions: LayoutDimension {
    definition {
        Physical dimensions of the rectangular bounding box for the rendered Line.
    }
    }
    }
    association {
        owner_type inheritance
        owner LayoutLine
        member_type direct
        member DisplayLine
    }
    association {
        owner_type inheritance
        owner LayoutLine
        member_type direct
        member FormLine
    }
    }
}

```

Figure 11

16 Display template

```

diagram DisplayTemplate {
    title 'Display template'
    caption 'Display template data model'
    association {
        owner TextElement
        member_type direct
        member DisplayLine
    }
    class DisplayTemplate {
        definition {
            The display template is used for displaying an
            interchange address instance belonging to an interchange

```

address class. Generally, one address class is represented by one display template.

```

}
}
association {
    owner DisplayTemplate
    member_type inheritance
    member LayoutTemplate
}
association {
    owner_type composition
    owner DisplayTemplate
    member_type direct
    member DisplayLine#formLine [1..*]
}
class DisplayLine {
    definition {
A display line represents a line within a display layout.
}
}
association {
    owner_type composition
    owner DisplayLine
    member_type direct
    member DisplayLineElement#element [1..*]
}
class DisplayLineElement {
    definition {
This represents an element within a display line.
}
}
class DataElement {
    definition {
Representing a variable data value that is contained in the interchange address
component, such as the PO box number following the phrase "PO Box".
}
}
+value: CharacterString {
    definition {
Representing a variable data value that is contained in the interchange address
component, such as the PO box number following the phrase "PO Box". This element
indicates that a form field is required, and the data values accepted shall adhere to
the data value type defined in its associated interchange address component (through
`FormLine`).
}
}
}
association {
    owner DataElement
    member_type inheritance
    member DisplayLineElement
}
association {
    owner DataElement
    member_type inheritance
    member FormLineElement
}
association {
    owner_type inheritance
    owner DataElement
    member_type direct
    member DataElementWithConditions
}
class TextElement {

```

```

    definition {
        Representing static text, such as the phrase "PO Box" preceding the actual PO box
        number, for display layouts for PO boxes.
    }
    +text: CharacterString {
        definition {
            The actual value of the TextElement.
        }
    }
    association {
        owner TextElement
        member_type inheritance
        member DisplayLineElement
    }
}

```

Figure 12

17 Form template

```

diagram FormTemplate {
    title 'Form template'
    caption 'Form template data model'
    association {
        owner FormTemplate
        member_type direct
        member FormLineElement
    }
    association {
        owner FormLineElement
        member_type direct
        member StaticTextElement
    }
    association {
        owner DataElement
        member_type direct
        member StaticTextElement
    }
    class FormTemplate {
        definition {

```

This section shows how an input form can be rendered according to the interchange address class.

The form template should use should also provide an example for display purposes in input (e.g., Help section).

The form template is used for displaying an input form for entry of address data into an interchange address instance.

Generally, one address class is represented by one input template.

NOTE: This part has no equivalence in PATDL.

```

    }
    }
    association {
        owner FormTemplate
        member_type inheritance
        member LayoutTemplate
    }
    association {
        owner_type composition
        owner FormTemplate
        member_type direct
        member FormLine#formLine [1..*]
    }
}

```

```

}
class FormLine {
  definition {
    A form line represents a line within a form layout.
  }

```

It extends from the `LineElementReference` model, and takes a set of elements that contain form line element references (`FormLineElementReference`).

```

}
}
association {
  owner_type composition
  owner FormLine
  member_type direct
  member FormLineElement#element [1..*]
}
class FormLineElement {
  definition {
    This represents an element within a form line.
  }

```

In an empty form with fields generated from data elements, if the `example` values of the associated interchange address component are specified, such values should be used as placeholders for form input.

```

}
}
class SelectionPair {
  definition {
    Representing a key value pair of a selection, e.g. in the case of the US
    States, "Rhode Island" is represented as the displayAs attribute and the
    postal code "RI" is represented as the key attribute.
  }
}
+key: CharacterString {
  definition {
    The mapping key as a unique identifier of the key value pair of this SelectionPair.
  }
}
+displayAs: CharacterString {
  definition {
    The display value of the key value pair of this SelectionPair.
  }
}
}
class SelectionElement {
  definition {
    Representing a selectable mapping between display string to code, e.g. in the case
    of the US States, "Rhode Island" the state name is mapped to the "RI" postal code.
    Represented by `SelectionElement` and within `SelectionPair`.
  }
}
+map: SelectionPair [1..*]{
  definition {
    Mapping between a display string to code for each instance of SelectionPair.
  }
}
}
association {
  owner SelectionElement
  member_type inheritance
  member FormLineElement
}
class DataElement {
  definition {
    Representing a variable data value that is contained in the interchange address
    component, such as the PO box number following the phrase "PO Box".
  }

```

```

}
    +value: CharacterString {
    definition {
        Representing a variable data value that is contained in the interchange address
        component, such as the PO box number following the phrase "PO Box". This element
        indicates that a form field is required, and the data values accepted shall adhere to
        the data value type defined in its associated interchange address component (through
        `FormLine`).
    }
}
}
    association {
        owner DataElement
        member_type inheritance
        member DisplayLineElement
    }
    association {
        owner DataElement
        member_type inheritance
        member FormLineElement
    }
    association {
        owner_type inheritance
        owner DataElement
        member_type direct
        member DataElementWithConditions
    }
    class StaticTextElement {
    definition {
        Static text that can be a constituent part of an `ProfileCompliantAddress` that complies
        with an `InterchangeAddressClassProfile`.
    }
    +value: CharacterString {
    definition {
        Representing static text, such as the phrase "`PO Box`" preceding the actual PO box
        number, for form layouts for PO boxes. This element is meant to be shown in the form but
        not allow modification.
    }
}
}
    association {
        owner StaticTextElement
        member_type inheritance
        member FormLineElement
    }
    class DataElementWithConditions {
    definition {
        DataElementWithConditions extends from DataElement to represent DataElement with
        additional constraints.
    }
    +value: iso19115MD_Constraints {
    definition {
        The constraint information of `MD_Constraint` as defined in <<ISO19115-1>>.
    }
}
}
}
}

```

Figure 13

Annex A (informative)

Abstract test suites

A.1 Introduction

The abstract test suites for the conformance classes defined by this part of ISO 19160 are presented in [A.3](#) to [A.10](#).

A.2 Conformance class: Address profile register

Refer to ISO 19135-1:2015 for requirements.

A.3 Conformance class: AddressProfile

Table A.1 — AddressProfile test 1: Associations

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 10
Test type	Basic

Table A.2 — AddressProfile test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 10
Test type	Basic

A.4 Conformance class: AddressClassProfile

Table A.3 — AddressClassProfile test 1: Associations

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 10
Test type	Basic

Table A.4 — AddressClassProfile test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 10

Table A.4 — AddressClassProfile test 2: Attributes*(continued)*

Test type	Basic
-----------	-------

A.5 Conformance class: AddressComponentProfile**Table A.5 — AddressComponentProfile test 1: Associations**

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 10
Test type	Basic

Table A.6 — AddressComponentProfile test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 10
Test type	Basic

A.6 Conformance class: ProfileCompliantAddress**Table A.7 — ProfileCompliantAddress test 1: Associations**

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 11
Test type	Basic

Table A.8 — ProfileCompliantAddress test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 11
Test type	Basic

A.7 Conformance class: ProfileCompliantAddressComponent**Table A.9 — ProfileCompliantAddressComponent test 1: Associations**

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 11
Test type	Basic

Table A.10 — ProfileCompliantAddressComponent test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 11
Test type	Basic

A.8 Conformance class: InterchangeAddressClassProfile

Table A.11 — InterchangeAddressClassProfile test 1: Associations

Test purpose	Check that the model contains the associations as specified.
Test method	Inspect the model
Reference	Clause 12
Test type	Basic

Table A.12 — InterchangeAddressClassProfile test 2: Attributes

Test purpose	For each instance in the class, check that the instance appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 12
Test type	Basic

A.9 Conformance class: FormTemplate

Table A.13 — FormTemplate test 1: Attributes

Test purpose	For each class and type in the model, check that the model appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 17
Test type	Basic

A.10 Conformance class: DisplayTemplate

Table A.14 — DisplayTemplate test 1: Attributes

Test purpose	For each class and type in the model, check that the model appropriately includes the mandatory, optional and conditional attributes.
Test method	Inspect the model
Reference	Clause 16
Test type	Basic

Annex B (informative)

Usage

B.1 Accuracy and verification

An authority, such as the local post office, could “verify” a structured address that it is confirmed that this address instance is a “deliverable address” through an `AddressFeature`.

The owner of the address, such as the tenant of an office, could provide its signed, structured address on an electronic business card. This allows the recipient of the business card to know whether the senders address is authentic. If this address is verified to be a “deliverable address”, the recipient will know that items sent to this address will very likely be deliverable.

Extra steps need to be taken here to allow this.

B.2 Address as identity

Addresses do not only specify a location, in some cases they are part of the identity. For example, in business cards, an address can mean more than just an address, such as with vanity value.

This document shall support this functionality for it to be useful in contact exchange.

This address represents a complete textual address instance.

EXAMPLE 1 — Fully expanded sample address Suite 1109, Floor 11, Central Building, 1-3 Pedder Street, Central, Central & Western District, Hong Kong Island, Hong Kong

And this address represents the identical address as above, with a reduction of information that is already implied, which that does not reduce its correctness. This can be achieved by supplying a `displayTemplate` in the interchange address.

EXAMPLE 2 — Shortened sample address Suite 1109, 1 Pedder Street, Central, Hong Kong

B.3 Address as destination

In certain cases, an address is expected to be reachable either by person and/or post.

An address instance should support being used in conjunction with routing information, acting as a waypoint, and/or supporting a source-defined route.

EXAMPLE Written instructions on how to deliver to a place can be unambiguous, but terribly difficult to locate. For instance, some buildings have split floors — rooms may have the same floor identifier, but is actually inaccessible from the same floor.

Annex C (informative)

Examples of objects specified in this document

Models specified in this document can be represented in various object structures, including in XML ISO/TS 19139:2007 and in JSON.

C.1 AddressProfile

```
{
  "id": "http://standards.isotc211.org/19160/-6/tc211-sample.adp",
  "type": "iso-19160-address-profile",
  "publisher": "http://standards.isotc211.org/19160/-6/",
  "signature": "...",

  "name": "TC 211 Minimal Address Profile",
  "localization": {
    "language": "en",
    "script": "Latn"
  },

  "area": {
    "countries": ["uk"]
  },

  "valueTypes": {
    "addressedObjectIdentifier": {
      "primitiveType": "Integer"
    }
  },
  "addressComponents": { ... },
  "addressClassProfiles": { ... }
}
```

Figure C.1

C.2 AddressClassProfiles

```
"addressClassProfiles": {
  "streetAddress": {
    "description": "Street Address",
    "addressComponentSpecifications": [ (AddressComponentSpecification) 1..* ... ]
    "displayTemplate": { ... },
    "formTemplate": { ... }
  }
}
```

Figure C.2

C.3 Validity

```
"validity": {
  "validFrom": "20171129Z000000",
  "validTo": "20191129Z000000"
}
```

```
}
```

Figure C.3**C.4 PublisherInformation**

```
"publisher": {
  "publisherName": "UK Post Office",
  "publisherUri": "https://www.postoffice.co.uk"
}
```

Figure C.4**C.5 Localization**

```
"localization": {
  "language": "en",
  "script": "Latn"
}
```

Figure C.5**C.6 Signature**

```
"signature": {
  "algorithm": "1.2.3.4.5.6.7.8.9",
  "publicKey": "https://www.postoffice.co.uk/profile-signature.key",
  "signature": "BOLVMNoGNM1TLglnlxgm0a9t"
}
```

Figure C.6**C.7 AddressClassProfile**

```
"addressClassProfile": {
  "id": "streetAddress",
  "description": "A typical street address",
  "addressComponentSpecifications": [ ... ],
  "displayTemplate": { ... },
  "formTemplate": { ... },
}
```

Figure C.7**C.8 User Defined Data Types**

```
"valueTypes": [{
  "name": "addressNumberValue",
  "coreType": "Integer",
  "constraints": [ ... ],
}]
```

Figure C.8**C.9 Data Type Constraints**

```
"constraints": [{
  "maxValue": 10000,
  "minValue": 1
}]
```

```
}]
```

Figure C.9

C.10 AddressComponentClass

```
"addressComponentClass": {  
  "key": "addressNumber",  
  "description": "Street number",  
  "datatype": "addressNumberValue"  
}
```

Figure C.10

C.11 ProfileCompliantAddress

```
"address": {  
  "profileId": "https://standards.iso/211.org/19160/-6/profiles/uk.adp",  
  "components": [ ... ],  
  "signature": { ... },  
  "features": [ ... ]  
}
```

Figure C.11

C.12 AddressComponent

```
"addressComponent": {  
  "type": "addressNumber",  
  "values": [ 1001 ]  
}
```

Figure C.12

C.13 AddressFeature

```
"addressFeature": {  
  "feature": "https://standards.iso/211.org/19160/-6/features/specified",  
  "signature": [ ... ]  
}
```

Figure C.13

C.14 DisplayTemplate

```
"DisplayTemplate": {  
  ...  
}
```

Figure C.14

C.15 FormTemplate

```
"FormTemplate": {  
  ...  
}
```

Figure C.15

Annex D (informative)

Examples

D.1 Example of address profiles defined in ISO 19160-1

D.1.1 ISO 19160-1 C.2 Example 1: Minimal address profile

D.1.1.1 Address profile (JSON)

```
{
  "id": "http://standards.isotc211.org/19160/-6/tc211-minimal.adp",
  "type": "iso-19160-address-profile",
  "publisher": "http://standards.isotc211.org/19160/-6/",
  "signature": "...",

  "name": "TC 211 Minimal Address Profile",
  "locale": {
    "language": "en",
    "script": "Latn"
  },

  "addressComponents": {
    "addressLine": {
      "dataType": "CharacterString",
      "minCardinality": 1
    },
  },

  "addressClassProfiles": {
    "minimalAddress": {
      "addressComponentSpecifications": [
        {
          "componentType": "addressLine",
          "description": "Address line",
          "require": true
        }
      ],
      "displayTemplates": [
        {
          "id": "basic",
          "name": "Basic",
          "description": "Rendering of the minimal address profile",
          "orientation": "horizontal",
          "localization": {
            "script": "Latn",
            "language": "en"
          },
          "lines": [
            {
              "componenet": "addressLine",
              "cardinality": "*"
            }
          ]
        }
      ],
      "formTemplates": [
        {

```

```
{
  "id": "basic",
  "name": "Basic",
  "description": "Address input form for the minimal address profile",
  "orientation": "horizontal",
  "localization": {
    "script": "Latn",
    "language": "en"
  },
  "formLines": [
    {
      "fields": [
        { "component": "addressLine" }
      ],
      "cardinality": "*"
    }
  ]
}
```

Figure D.1

D.1.1.2 Profile-compliant address (ISO 19160-1, Figure C.3, South African street address) (JSON)

```
{
  "profile": "http://standards.iso/2011.org/19160/-6/tc211-minimal.adp",
  "components": [
    {
      "type": "addressLine",
      "value": "14 Church Street"
    },
    {
      "type": "addressLine",
      "value": "Hatfield"
    },
    {
      "type": "addressLine",
      "value": "South Africa"
    }
  ]
}
```

Figure D.2

D.1.1.3 Profile-compliant address (ISO 19160-1, Figure C.4, US landmark address) (JSON)

```
{
  "profile": "http://standards.iso/2011.org/19160/-6/tc211-minimal.adp",
  "components": [
    {
      "type": "addressLine",
      "value": "Statue of Liberty"
    },
    {
      "type": "addressLine",
      "value": "Liberty Island"
    },
    {
      "type": "addressLine",
      "value": "New York"
    },
    {
      "type": "addressLine",

```

```

        "value": "NY"
    }
]
}

```

Figure D.3**D.1.1.4 Profile-compliant address (ISO 19160-1, Figure C.5, UK address) (JSON)**

```

{
  "profile": "http://standards.iso/19160/-6/tc211-minimal.adp",
  "components": [
    {
      "type": "addressLine",
      "value": "Multi-storey car park at Southampton Magistrates Court"
    },
    {
      "type": "addressLine",
      "value": "Carlton Crescent"
    },
    {
      "type": "addressLine",
      "value": "Southampton, Bevois"
    },
    {
      "type": "addressLine",
      "value": "SO17 1EY"
    },
    {
      "type": "addressLine",
      "value": "United Kingdom"
    }
  ]
}

```

Figure D.4**D.1.2 ISO 19160-1 C.3 Example 2: Sample address profile****D.1.2.1 Address profile (JSON)**

ISO 19160-1, Table C.4 (Address component type) is also incorporated in this profile.

```

{
  "id": "http://standards.iso/19160/-6/tc211-sample.adp",
  "type": "iso-19160-address-profile",
  "publisher": "http://standards.iso/19160/-6/",
  "signature": "...",

  "name": "TC 211 Minimal Address Profile",
  "locale": {
    "language": "en",
    "script": "Latn"
  },

  "valueTypes": {
    "addressedObjectIdentifierValue": {
      "primitiveType": "Integer",
      "maxValue": 10000,
      "minValue": 1
    },
    "thoroughfareNameValue": {
      "type": "Object",
      "value": {
        "prefix": {

```

```

        "primitiveType": "CharacterString"
    },
    "name": {
        "primitiveType": "CharacterString"
    },
    "suffix": {
        "primitiveType": "CharacterString"
    },
    "type": {
        "primitiveType": "CharacterString"
    }
}
}
}

"addressComponents": {
    "addressNumber": {
        "dataType": "addressedObjectIdentifierValue",
        "description": "Address number"
    },
    "boxNumber": {
        "dataType": "addressedObjectIdentifierValue",
        "description": "PO Box number"
    },
    "thoroughfareName": {
        "dataType": "thoroughfareNameValue",
        "description": "Thoroughfare name"
    },
    "localityName": {
        "dataType": "CharacterString",
        "description": "Locality name"
    },
    "postOfficeName": {
        "dataType": "CharacterString",
        "description": "Post office name"
    },
    "postCode": {
        "dataType": "CharacterString",
        "description": "Post code"
    },
    "countryName": {
        "dataType": "CharacterString",
        "description": "Country"
    }
},

"addressClassProfiles": {

    "streetAddress": {
        "description": "Street Address",
        "addressComponentSpecifications": [
            {
                "componentType": "addressNumber",
                "minCardinality": 1,
                "maxCardinality": 1,
                "required": true
            },
            {
                "componentType": "thoroughfareName",
                "minCardinality": 1,
                "maxCardinality": 1,
                "required": true
            },
            {
                "componentType": "placeName",
                "dataType": "CharacterString",

```

```

        "minCardinality": 1,
        "maxCardinality": 1,
        "required": true
    },
    {
        "componentType": "postCode",
        "minCardinality": 1,
        "maxCardinality": 1,
        "required": true
    },
    {
        "componentType": "countryName",
        "minCardinality": 1,
        "maxCardinality": 1,
        "required": false,
    },
],
"displayTemplates": [
    {
        "id": "basic",
        "name": "Basic",
        "description": "Rendering of the minimal address profile",
        "orientation": "horizontal",
        "localization": {
            "script": "Latn",
            "language": "en"
        },
        "lines": [
            {
                "type": "template",
                "value": "{{ addressNumber }} {{ thoroughfareName.name }}"
            },
            {
                "type": "template",
                "value": "{{ placeName }} {{ postCode }}"
            },
            {
                "type": "component",
                "value": "countryName"
            }
        ]
    }
],
"formTemplates": [
    {
        "id": "basic",
        "name": "Basic",
        "description": "Address input form for the sample address profile",
        "orientation": "horizontal",
        "localization": {
            "script": "Latn",
            "language": "en"
        },
        "formLines": [
            {
                "fields": [
                    { "component": "addressNumber" },
                    { "component": "thoroughfareName" }
                ]
            },
            {
                "fields": [
                    { "component": "placeName" },
                    { "component": "postCode" }
                ]
            }
        ]
    }
]

```

```

        ]
      },
      {
        "fields": [
          { "component": "countryName" }
        ]
      }
    ]
  }
]
},
"boxAddress": {
  "description": "Box Address",
  "addressComponentSpecifications": [
    {
      "componentType": "boxNumber",
      "minCardinality": 1,
      "maxCardinality": 1,
      "required": true
    },
    {
      "componentType": "postOfficeName",
      "dataType": "CharacterString",
      "minCardinality": 1,
      "maxCardinality": 1,
      "required": true
    },
    {
      "componentType": "postCode",
      "minCardinality": 1,
      "maxCardinality": 1,
      "required": true
    },
    {
      "componentType": "countryName",
      "minCardinality": 1,
      "maxCardinality": 1,
      "required": false
    }
  ],
},
],
"displayTemplates": [
  {
    "id": "basic",
    "name": "Basic",
    "description": "Rendering of the minimal address profile",
    "orientation": "horizontal",
    "localization": {
      "script": "Latn",
      "language": "en"
    },
  },
  "lines": [
    {
      "type": "template",
      "value": "PO Box {{ boxNumber }}"
    },
    {
      "type": "component",
      "value": "postOfficeName"
    },
    {
      "type": "component",
      "value": "postCode"
    },
    {
      "type": "component",

```

```

    "value": "countryName"
  }
]
}
],

"formTemplates": [
{
  "id": "basic",
  "name": "Basic",
  "description": "Address input form for the minimal address profile",
  "orientation": "horizontal",
  "localization": {
    "script": "Latn",
    "language": "en"
  },
  "formLines": [
    {
      "fields": [
        { "component": "boxNumber" }
      ]
    },
    {
      "fields": [
        { "component": "postOfficeName" }
      ]
    },
    {
      "fields": [
        { "component": "postCode" }
      ]
    },
    {
      "fields": [
        { "component": "countryName" }
      ]
    }
  ]
}
]
}
}
}
```

Figure D.5

D.1.2.2 Profile-compliant address (representation of ISO 19160-1, Figure C.14, Street address) (JSON)

```
{
  "profile": "http://standards.isotc211.org/19160/-6/tc211-sample.adp#streetAddress",
  "components": [
    {
      "type": "addressNumber",
      "value": "99"
    },
    {
      "type": "thoroughfareName",
      "value": {
        "name": "Lombardy",
        "type": "Street"
      }
    },
    {
      "type": "placeName",
      "value": "The Hills"
    }
  ]
}
```

```
    },
    {
      "type": "postCode",
      "value": "0039"
    },
    {
      "type": "countryName",
      "value": "South Africa"
    }
  ]
}
```

Figure D.6

D.1.2.3 Profile-compliant address (representation of ISO 19160-1, Figure C.15, Box address) (JSON)

```
{
  "profile": "http://standards.iso/2011.org/19160/-6/tc211-sample.adp#boxAddress",
  "components": [
    {
      "type": "boxNumber",
      "value": "345"
    },
    {
      "type": "postOfficeName",
      "value": "Orlando"
    },
    {
      "type": "postCode",
      "value": "2020"
    },
    {
      "type": "countryName",
      "value": "South Africa"
    }
  ]
}
```

Figure D.7

Bibliography

- [1] ISO 639-3:2007, *Codes for the representation of names of languages — Part 3: Alpha-3 code for comprehensive coverage of languages*
- [2] ISO 3166-1:2020, *Codes for the representation of names of countries and their subdivisions — Part 1: Country code*
- [3] ISO/IEC 9798-1:2010, *Information technology — Security techniques — Entity authentication — Part 1: General*
- [4] ISO/IEC 9798-3:2019, *IT Security techniques — Entity authentication — Part 3: Mechanisms using digital signature techniques*
- [5] ISO/IEC/IEEE 9945:2009, *Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7*
- [6] ISO/IEC 10179:1996, *Information technology — Processing languages — Document Style Semantics and Specification Language (DSSSL)*
- [7] ISO/IEC 14888-3:2018, *IT Security techniques — Digital signatures with appendix — Part 3: Discrete logarithm based mechanisms*
- [8] ISO 15924:2004, *Information and documentation — Codes for the representation of names of scripts*
- [9] ISO 19136-1:2020, *Geographic information — Geography Markup Language (GML) — Part 1: Fundamentals*
- [10] ISO/TS 19139:2007, *Geographic information — Metadata — XML schema implementation*
- [11] ISO 19160-4:2017, *Addressing — Part 4: International postal address components and template language*
- [12] ISO/DIS 24229¹⁾, *Information and documentation — Codes for written language conversion systems*
- [13] ISO/IEC TR 29149:2012, *Information technology — Security techniques — Best practices for the provision and use of time-stamping services*
- [14] ISO/IEC/IEEE 31320-2:2012, *Information technology — Modeling Languages — Part 2: Syntax and Semantics for IDEF1X97 (IDEFobject)*

1) Under preparation. (Stage at the time of publication ISO/DIS 24229).

